

FRD

Functional Requirements Document (FRD)

AWS Glue ETL Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based ETL pipeline that ingests customer and order data from S3, performs data cleaning, implements SCD Type 2 logic using Apache Hudi, and produces analytical aggregations. The system will use AWS Glue Data Catalog for metadata management and support querying via Amazon Athena.

Scope

- In Scope:
 - Ingestion of customer and order CSV files from specified S3 locations
 - Data cleaning (null removal, deduplication)
 - Registration of datasets in AWS Glue Data Catalog
 - SCD Type 2 implementation for order summary using Apache Hudi
 - Incremental processing triggered by customer data changes
 - Aggregation of customer spending data
 - Monitoring via CloudWatch
- Out of Scope:
 - Real-time streaming ingestion
 - Data quality profiling beyond specified cleaning rules
 - User interface or reporting layer
 - Lake Formation governance implementation (optional component)
 - Data archival or retention policies

Assumptions

1. CSV files in source S3 locations follow consistent formatting and are UTF-8 encoded
2. CustId is the primary key for customer data and OrderId is the primary key for order data
3. "Null" (string) and NULL (actual null) are the only null representations requiring removal
4. Duplicate detection is based on all columns in each dataset
5. The Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` already exists

6. AWS Glue job has appropriate IAM permissions for S3 read/write and Glue Catalog access
7. Date fields in Order data are in a parseable format (specific format not provided in BRD)
8. Price calculations use standard arithmetic (PricePerUnit Qty)
9. SCD Type 2 changes are detected by comparing all customer attributes except CustId
10. Incremental processing frequency and trigger mechanism are defined at implementation time
11. CloudWatch logging is enabled by default for Glue jobs
12. Athena queries will be executed by end users outside this pipeline scope
-

2. Functional Requirements

FR-INGEST-001: Customer Data Ingestion

- Title: Ingest Customer CSV Data from S3
- Description:

The system shall read customer data in CSV format from the specified S3 location and load it into memory for processing.

- Business Objective:

Enable the ETL pipeline to access source customer information for downstream processing and analysis.

- Inputs:
 - Source: s3://adif-sdlc/sdlc_wizard/customerdata/
 - Format: CSV
 - Schema:
 - CustId: (datatype not specified in BRD)
 - Name: (datatype not specified in BRD)
 - EmailId: (datatype not specified in BRD)
 - Region: (datatype not specified in BRD)
- Processing Rules / Functional Steps:
 1. Connect to S3 location s3://adif-sdlc/sdlc_wizard/customerdata/
 2. Read all CSV files matching the customer data pattern
 3. Parse CSV with header row
 4. Load data into Spark DataFrame for processing
- Outputs:
 - In-memory Spark DataFrame containing raw customer data

- Preconditions:
- S3 bucket and path exist and are accessible
- CSV files are present in the source location
- AWS Glue job has s3:GetObject permission on source bucket
- Postconditions:
- Customer data is available in memory for cleaning and transformation
- Error / Validation Conditions:
- If S3 path does not exist, job shall fail with clear error message
- If CSV parsing fails, job shall fail with file name and line number
- If no files are found, job shall fail indicating empty source
-

FR-INGEST-002: Order Data Ingestion

- Title: Ingest Order CSV Data from S3
- Description:

The system shall read order data in CSV format from the specified S3 location and load it into memory for processing.

- Business Objective:

Enable the ETL pipeline to access source order transaction information for downstream processing and analysis.

- Inputs:
- Source: s3://adif-sdlc/sdlc_wizard/orderdata/
- Format: CSV
- Schema:
- OrderId: (datatype not specified in BRD)
- ItemName: (datatype not specified in BRD)
- PricePerUnit: (datatype not specified in BRD)
- Qty: (datatype not specified in BRD)
- Date: (datatype not specified in BRD)
- CustId: (datatype not specified in BRD)
- Processing Rules / Functional Steps:
- 1. Connect to S3 location s3://adif-sdlc/sdlc_wizard/orderdata/
- 2. Read all CSV files matching the order data pattern

3. Parse CSV with header row

4. Load data into Spark DataFrame for processing

- Outputs:

- In-memory Spark DataFrame containing raw order data

- Preconditions:

- S3 bucket and path exist and are accessible

- CSV files are present in the source location

- AWS Glue job has s3:GetObject permission on source bucket

- Postconditions:

- Order data is available in memory for cleaning and transformation

- Error / Validation Conditions:

- If S3 path does not exist, job shall fail with clear error message

- If CSV parsing fails, job shall fail with file name and line number

- If no files are found, job shall fail indicating empty source

-

FR-CLEAN-001: Data Cleaning for Customer and Order Datasets

- Title: Remove Null Values and Duplicates

- Description:

The system shall cleanse both customer and order datasets by removing records containing null values (both string "Null" and actual NULL) and eliminating duplicate records.

- Business Objective:

Ensure data quality by removing incomplete and redundant records before loading into catalog tables and performing transformations.

- Inputs:

- Raw customer DataFrame from FR-INGEST-001

- Raw order DataFrame from FR-INGEST-002

- Processing Rules / Functional Steps:

1. For each dataset (customer and order):

- Remove rows where any column contains the string value "Null" (case-sensitive)

- Remove rows where any column contains actual NULL values

- Remove duplicate rows based on all columns

2. Apply cleaning independently to customer and order datasets

3. Log count of records removed for monitoring purposes

- Outputs:

- Cleaned customer DataFrame

- Cleaned order DataFrame

- Preconditions:

- Raw data has been successfully ingested per FR-INGEST-001 and FR-INGEST-002

- Postconditions:

- All remaining records have no null values in any column

- No duplicate records exist in either dataset

- Error / Validation Conditions:

- If cleaning results in zero records, job shall log warning but continue

- If all records are removed, downstream processes may fail; this should be monitored

-

FR-CATALOG-001: Register Customer Table in Glue Data Catalog

- Title: Create and Populate sdhc_wizard_customer Catalog Table

- Description:

The system shall register the cleaned customer data as a table in the AWS Glue Data Catalog under the specified database and table name.

- Business Objective:

Make customer data discoverable and queryable through AWS Athena and other AWS analytics services.

- Inputs:

- Cleaned customer DataFrame from FR-CLEAN-001

- Schema:

- CustId

- Name

- EmailId

- Region

- Processing Rules / Functional Steps:

1. Write cleaned customer DataFrame to Glue Data Catalog

2. Target database: `gen\_ai\_poc\_databrickscoe`

3. Target table: `sdlc\_wizard\_customer`

4. Create table if it does not exist

5. Update table metadata if schema changes

- Outputs:

- Glue Data Catalog table: `gen\_ai\_poc\_databrickscoe.sdlc\_wizard\_customer`

- Table is queryable via Athena

- Preconditions:

- Database `gen\_ai\_poc\_databrickscoe` exists in Glue Data Catalog

- Cleaned customer data is available per FR-CLEAN-001

- Glue job has glue:CreateTable and glue:UpdateTable permissions

- Postconditions:

- Customer table exists in Glue Data Catalog

- Table metadata reflects current schema

- Data is accessible for querying

- Error / Validation Conditions:

- If database does not exist, job shall fail with clear error

- If table creation fails due to permissions, job shall fail

- Schema conflicts shall be logged and handled per Glue catalog update policies

-

FR-CATALOG-002: Register Order Table in Glue Data Catalog

- Title: Create and Populate sdlc_wizard_order Catalog Table

- Description:

The system shall register the cleaned order data as a table in the AWS Glue Data Catalog under the specified database and table name.

- Business Objective:

Make order data discoverable and queryable through AWS Athena and other AWS analytics services.

- Inputs:

- Cleaned order DataFrame from FR-CLEAN-001

- Schema:

- OrderId

- ItemName

- PricePerUnit
- Qty
- Date
- CustId

- Processing Rules / Functional Steps:
 1. Write cleaned order DataFrame to Glue Data Catalog
 2. Target database: `gen\_ai\_poc\_databrickscoe`
 3. Target table: `sdlc\_wizard\_order`
 4. Create table if it does not exist
 5. Update table metadata if schema changes

- Outputs:
 - Glue Data Catalog table: `gen\_ai\_poc\_databrickscoe.sdlc\_wizard\_order`
 - Table is queryable via Athena

- Preconditions:
 - Database `gen\_ai\_poc\_databrickscoe` exists in Glue Data Catalog
 - Cleaned order data is available per FR-CLEAN-001
 - Glue job has glue:CreateTable and glue:UpdateTable permissions

- Postconditions:
 - Order table exists in Glue Data Catalog
 - Table metadata reflects current schema
 - Data is accessible for querying

- Error / Validation Conditions:
 - If database does not exist, job shall fail with clear error
 - If table creation fails due to permissions, job shall fail
 - Schema conflicts shall be logged and handled per Glue catalog update policies
-

FR-SCD2-001: Implement SCD Type 2 for Order Summary

- Title: Create and Maintain Order Summary with SCD Type 2 Logic Using Hudi
- Description:

The system shall join customer and order data, then implement Slowly Changing Dimension Type 2 logic to track historical changes in the ordersummary table. When customer data changes, new active rows shall be inserted and previous rows shall be closed with appropriate timestamps.

- Business Objective:

Maintain complete historical tracking of order summaries, preserving the state of customer information at the time of each order and tracking changes over time.

- Inputs:

- Cleaned customer DataFrame from FR-CLEAN-001
- Cleaned order DataFrame from FR-CLEAN-001
- Existing ordersummary Hudi table (for incremental updates)

- Processing Rules / Functional Steps:

1. Join customer and order DataFrames on CustId

2. For new records (not existing in ordersummary):

- Insert as new active rows
- Set IsActive = true
- Set StartDate = current timestamp
- Set EndDate = null
- Set OpTs = current timestamp

3. For existing records where customer attributes have changed:

- Close existing active row:
- Set IsActive = false
- Set EndDate = current timestamp
- Set OpTs = current timestamp
- Insert new active row with updated customer data:
- Set IsActive = true
- Set StartDate = current timestamp
- Set EndDate = null
- Set OpTs = current timestamp

4. Write results to Hudi table format at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

5. Update Glue Data Catalog table `ordersummary`

- Outputs:

- Hudi table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Glue Data Catalog table: `gen\_ai\_poc\_databrickscoe.ordersummary`
- SCD Type 2 columns: IsActive, StartDate, EndDate, OpTs

- Preconditions:

- Cleaned customer and order data available per FR-CLEAN-001

- S3 location s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ is accessible
- Glue job has s3:PutObject permission on target location
- Hudi libraries are available in Glue job environment
- Postconditions:
 - All current records have IsActive = true and EndDate = null
 - All historical records have IsActive = false and EndDate populated
 - Complete audit trail of changes is maintained
 - Table is queryable via Athena with time-travel capability
- Error / Validation Conditions:
 - If join results in zero records, log warning and create empty target table
 - If Hudi write fails, job shall fail and rollback
 - If multiple active rows exist for same business key, data integrity error shall be raised
 - If customer data contains no CustId, join will exclude those records
-

FR-INCR-001: Incremental Processing Based on Customer Updates

- Title: Trigger SCD Type 2 Updates When Customer Data Changes
- Description:

The system shall detect changes in customer data and trigger incremental updates to the ordersummary table, applying SCD Type 2 logic only to affected records.

- Business Objective:

Optimize processing performance by updating only changed records rather than reprocessing entire datasets, while maintaining complete historical accuracy.

- Inputs:
 - New/updated customer data from s3://adif-sdlc/sdlc_wizard/customerdata/
 - Existing customer data from previous run
 - Existing ordersummary Hudi table
- Processing Rules / Functional Steps:
 1. Compare incoming customer data with existing customer data
 2. Identify changed records (any attribute except CustId differs)
 3. Identify new customer records (CustId not in existing data)
 4. For changed customers:
- Retrieve all related order records

- Apply SCD Type 2 logic per FR-SCD2-001 to affected ordersummary rows

5. For new customers:

- Process all their orders as new records per FR-SCD2-001

6. Maintain SCD Type 2 metadata columns: IsActive, StartDate, EndDate, OpTs

- Outputs:

- Updated ordersummary Hudi table with incremental changes
- Log of processed customer changes

- Preconditions:

- Initial full load of ordersummary has been completed
- Customer data from previous run is available for comparison
- FR-SCD2-001 logic is implemented and functional

- Postconditions:

- Only affected ordersummary records are updated
- Historical integrity is maintained
- IsActive, StartDate, EndDate, OpTs are correctly maintained

- Error / Validation Conditions:

- If change detection fails, fall back to full reprocessing
- If no changes detected, skip ordersummary update
- If incremental update fails, job shall fail with clear error message

-

FR-AGG-001: Generate Customer Aggregate Spend Table

- Title: Create Customer Aggregate Spend Summary

- Description:

The system shall aggregate order data by customer to calculate total spending and produce a summary table for analytical purposes.

- Business Objective:

Provide business users with quick access to customer spending metrics for analysis and reporting.

- Inputs:

- Cleaned customer DataFrame from FR-CLEAN-001
- Cleaned order DataFrame from FR-CLEAN-001
- Derived calculation: TotalAmount = PricePerUnit Qty (per order)

- Processing Rules / Functional Steps:

1. Join customer and order data on CustId
2. Calculate TotalAmount for each order: PricePerUnit Qty
3. Group by customer Name and Date
4. Sum TotalAmount for each customer-date combination
5. Select columns: Name, TotalAmount, Date
6. Write results to s3://adif-sdlc/analytics/customeraggregatespend/
7. Register as Glue Data Catalog table `customeraggregatespend`

- Outputs:

- Aggregated data at s3://adif-sdlc/analytics/customeraggregatespend/
- Glue Data Catalog table: `gen\_ai\_poc\_databrickscoe.customeraggregatespend`

- Schema:

- Name
- TotalAmount
- Date

- Preconditions:

- Cleaned customer and order data available per FR-CLEAN-001
- S3 location s3://adif-sdlc/analytics/customeraggregatespend/ is accessible
- Glue job has s3:PutObject permission on target location

- Postconditions:

- Aggregated spending data is available for querying
- Table reflects current state of customer spending by date
- Data is accessible via Athena

- Error / Validation Conditions:

- If join results in zero records, create empty target table
- If PricePerUnit or Qty are non-numeric, exclude those records and log warning
- If aggregation fails, job shall fail with clear error message
- Null values in Name or Date shall exclude those records from aggregation

-

3. Non-Functional Considerations

Monitoring and Observability

- All AWS Glue jobs shall log execution metrics to CloudWatch

- Job success/failure status shall be logged
- Record counts at each processing stage shall be logged
- Processing duration shall be tracked

Performance

- Glue jobs shall leverage Spark parallelism for distributed processing
- Hudi table format shall enable efficient incremental updates
- Partitioning strategy for large datasets may be implemented at technical design phase

Scalability

- Solution shall handle growing data volumes through Glue's auto-scaling capabilities
- S3 storage shall accommodate increasing data retention requirements

Security

- All data in S3 shall be encrypted at rest
- IAM roles shall follow principle of least privilege
- Glue Data Catalog access shall be controlled via IAM policies

Availability

- Glue jobs shall be configured with retry logic for transient failures
- S3 provides 99.999999999% durability for stored data
-

4. Requirement Traceability Notes

Requirement ID	BRD Section	Key Traceability Notes	
-----	-----	-----	
FR-INGEST-001	Source Input (AWS S3), Schemas	Traces to customer CSV source and schema definition	
FR-INGEST-002	Source Input (AWS S3), Schemas	Traces to order CSV source and schema definition	
FR-CLEAN-001	Data Cleaning	Traces to null removal and deduplication requirements	
FR-CATALOG-001	Glue Catalog Tables	Traces to sdlc_wizard_customer table requirement	
FR-CATALOG-002	Glue Catalog Tables	Traces to sdlc_wizard_order table requirement	
FR-SCD2-001	SCD Type 2 (Hudi on S3)	Traces to ordersummary table, Hudi format, and SCD2 logic	
FR-INCR-001	Incremental Logic	Traces to customer update triggers and SCD2 maintenance	
FR-AGG-001	Aggregation Table	Traces to customeraggregatespend table and aggregation logic	

- Cross-Requirement Dependencies:
- FR-CATALOG-001 and FR-CATALOG-002 depend on FR-CLEAN-001
- FR-SCD2-001 depends on FR-CLEAN-001

- FR-INCR-001 depends on FR-SCD2-001
- FR-AGG-001 depends on FR-CLEAN-001
- Source Data Traceability:
 - All source data originates from s3://adif-sdlc/sdlc_wizard/customerdata/ and s3://adif-sdlc/sdlc_wizard/orderdata/
 - All outputs are written to s3://adif-sdlc/curated/ or s3://adif-sdlc/analytics/ paths
 - All catalog tables reside in database: gen_ai_poc_databrickscoe
 -
- Document Control:
 - This FRD is derived exclusively from the provided BRD
 - All assumptions are clearly documented in Section 1
 - All requirements are traceable to BRD sections
 - Technical implementation details are deferred to the TRD phase